第一部分： Android内存管理机制回顾

Android内存管理机制回顾-----ADJ

在 Android 的 lmk 机制中，会对于所有进程进行分类，对于每一类别的进程会有其 oom_adj 值的取值范围，**oom_adj 值越高则代表进程越不重要**，在系统执行低杀操作时，会从 oom_adj 值越高的开始杀。进程级别以变量的形式定义在 ProcessList.java 中。

从 Android 7.0 开始，ADJ 采用 100、200、300。在这之前的版本 ADJ 采用数字 1、2、3。这样的调整可以更进一步地细化进程的优先级。

```
adb shell
cd /proc/pid/
oom_adj   oom_score  oom_score_adj
cat oom_adj
0 表示前台进程FOREGROUND_APP_ADJ
```

Android 7.0之前的版本：
oom_score - - 是该进程的最终得分，分数越高，越容易被杀死；
oom_score_adj - - 范围：[-1000, 1000]，是kernel用来配置进程优先级的。值越低，最终的oom_score越低。
oom_adj - - 范围：[-16, 15]，是给用户来配置进程优先级的。为了方便用户配置，提供了范围更小的oom_adj参数，数字越小优先级越高，-17表示该进程被保护，不被OOMKiller杀死。
kernel会转换并更新该进程实际的oom_score_adj值，它们的换算关系是：

```
#define OOM_DISABLE (-17)
    #define OOM_SCORE_ADJ_MAX 1000

    oom_score_adj = (oom_adj * OOM_SCORE_ADJ_MAX) / -OOM_DISABLE;
    oom_adj = (oom_score_adj * - OOM_DISABLE) / OOM_SCORE_ADJ_MAX;
```

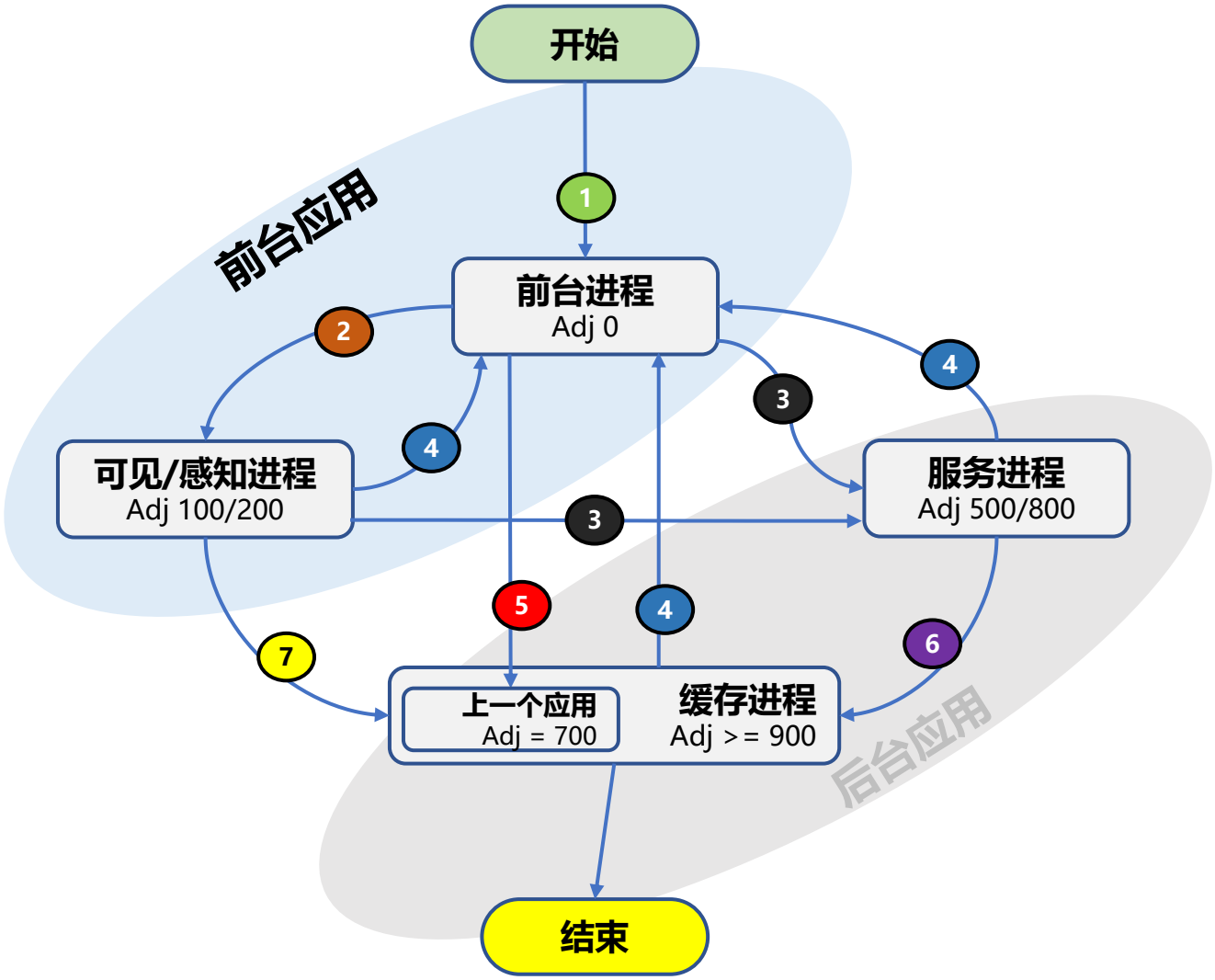
Android 7.0之前的版本，oom_score_adj= oom_adj * 1000/17； 而Android 7.0开始，oom_score_adj= oom_adj，不用再经过一次转换。

查看优先级adb shell dumpsys meminfo

PERCEPTIBLE_LOW_APP_ADJ 为 Android 10 新增；
PERCEPTIBLE_RECENT_FOREGROUND_APP_ADJ 为 Android 9 新增。

ADJ	取值	含义	缩写
NATIVE_ADJ	-1000	native 进程	ntv
SYSTEM_ADJ	-900	system 进程默认值	system
PERSISTENT_PROC_ADJ	-800	持久进程，例如通话	pers
PERSISTENT_SERVICE_ADJ	-700	系统进程或持久进程绑定的进程	psvc
FOREGROUND_APP_ADJ	0	前台进程	fg
PERCEPTIBLE_RECENT_FOREGROUND_APP_ADJ	50	最近从 TOP 转移到 FGS 状态的进程，继续像前台应用一样对待	
VISIBLE_APP_ADJ	100	仅托管可见 Activity 的进程	vis
PERCEPTIBLE_APP_ADJ	200	仅托管用户可感知组件的进程，例如后台音乐播放	prcp
PERCEPTIBLE_LOW_APP_ADJ	250	被系统或其他 app 绑定的进程，比 Service 重要，用户可感知	prcl
BACKUP_APP_ADJ	300	备份进程	bkup
HEAVY_WEIGHT_APP_ADJ	400	重量级进程	hvy
SERVICE_ADJ	500	服务进程，A List	svc
HOME_APP_ADJ	600	Launcher 进程	home
PREVIOUS_APP_ADJ	700	前一个应用进程	prev
SERVICE_B_ADJ	800	服务进程的 B List，没有 A List 重要	svcb
CACHED_APP_LMK_FIRST_ADJ	950	优先被杀死的级别	cch
CACHED_APP_MIN_ADJ	900	仅托管不可见 Activity 的进程	cch
CACHED_APP_MAX_ADJ	906	同上	cch
UNKNOWN_ADJ	1001	未知的 ADJ	

Android内存管理机制回顾-----进程状态转换

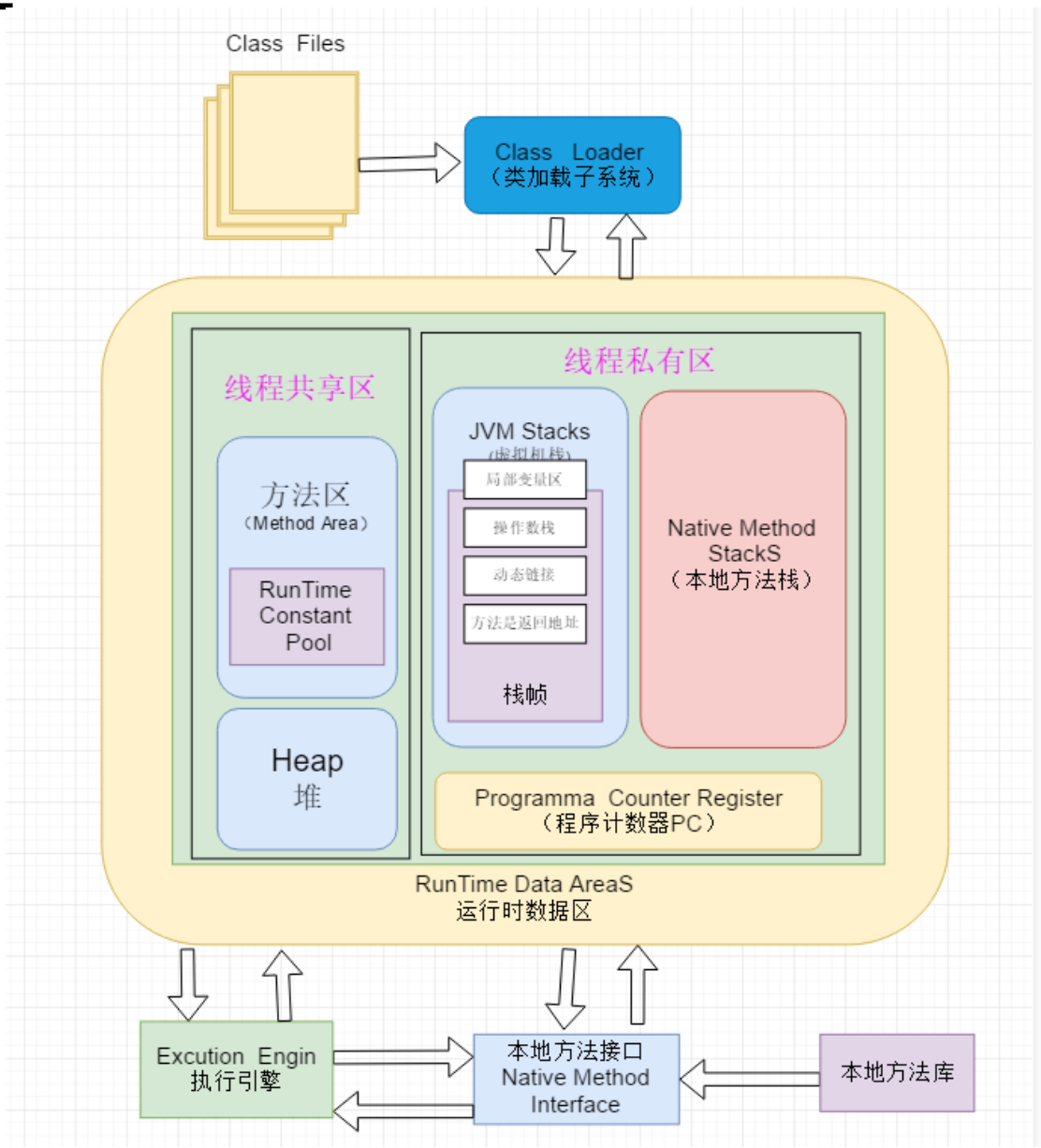


切换事件	事件描述
1	1、用户在桌面启动应用 2、通过应用跳转拉起应用 3、在多任务卡片启动最近运行过的任务
2	1、应用被弹框遮挡 2、应用启动了ForegroundService，并将应用切换到后台（HOME键、返回、应用跳转等）
3	1、应用启动了Service组件，并将应用切换到后台（HOME键、返回、应用跳转等）
4	1、用户通过任务卡片恢复 2、用户重新点击启动图标 3、应用运行跳转到前台界面 4、在通知栏上点击运行的服务通知跳转 5、覆盖的应用窗口的对话框被Dismiss
5	**1、用户将应用A切换到后台（HOME键、返回、应用跳转等），再启动其他应用B的Activity。** 应用A先变为上一个应用，等再启动另一个应用C后，应用A变为缓存进程；应用B变为上一个应用。
6	1、应用的服务组件（Service组件）运行停止或结束 2、Service运行超过20小时自动进入
7	1、被遮挡场景。用户将应用完全切换到后台（HOME键、返回、应用跳转等），再启动其他应用的Activity 2、FGS场景。直接停止应用中的Foreground Service

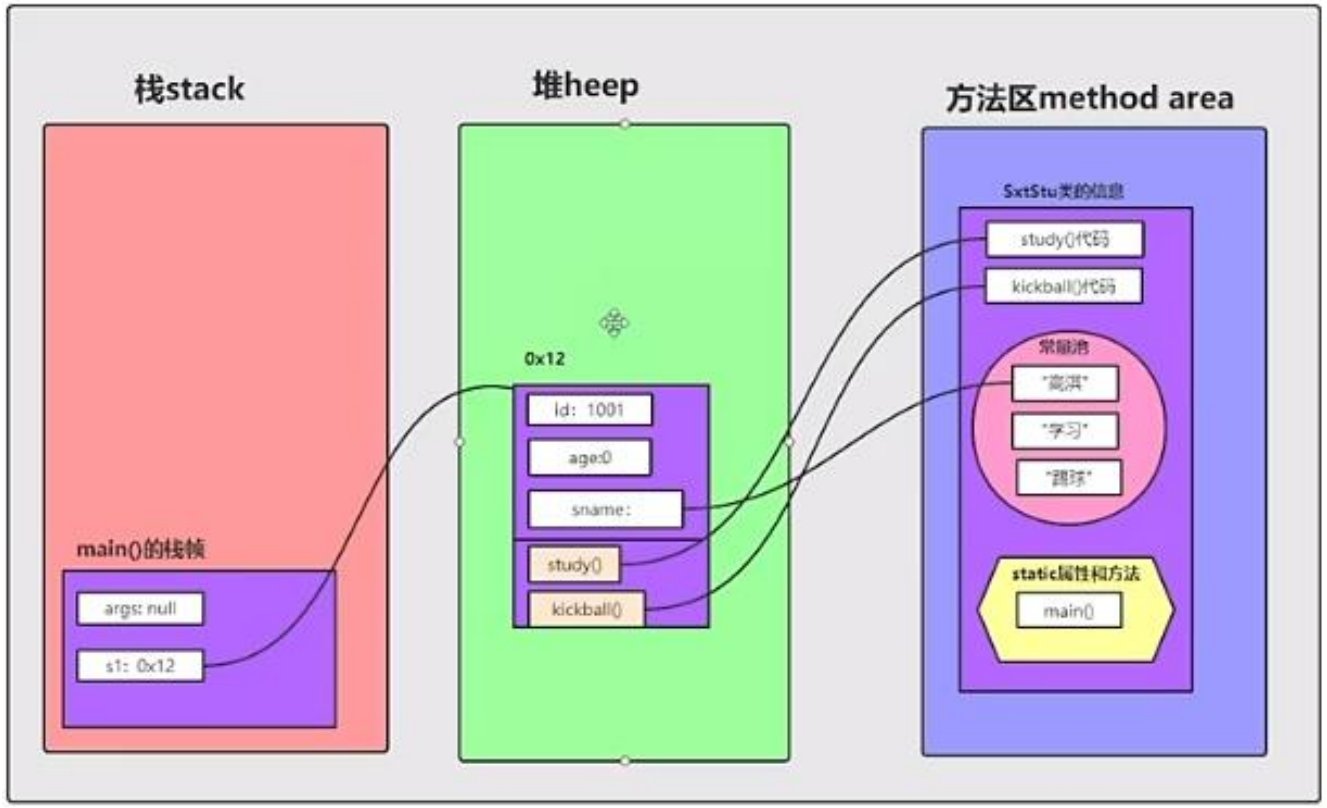
多任务场景下，前台进程切换到后台缓存进程，可能被系统查杀，切换到前台无法恢复状态

Android内存管理机制回顾-----内存模型

方法区
•线程共享区域，用于存储类信息、静态变量、常量、即时编译器编译出来的代码数据。
•无法满足内存分配需求时会发生OOM。
堆
•线程共享区域，是JAVA虚拟机管理的内存中最大的一块，在虚拟机启动时创建。
•存放对象实例，几乎所有的对象实例都在堆上分配，GC管理的主要区域。
虚拟机栈
•线程私有区域，每个java方法在执行的时候会创建一个栈帧用于存储局部变量表、操作数栈、动态链接、方法出口等信息。方法从执行开始到结束过程就是栈帧在虚拟机栈中入栈出栈过程。
•局部变量表存放编译期可知的基本数据类型、对象引用、returnAddress类型。所需的内存空间会在编译期间完成分配，进入一个方法时在帧中局部变量表的空间是完全确定的，不需要运行时改变。
•若线程申请的栈深度大于虚拟机允许的最大深度，会抛出SatckOverFlowError错误。
•虚拟机动态扩展时，若无法申请到足够内存，会抛出OutOfMemoryError错误。
Native方法栈
•为虚拟机中Native方法服务，对本地方法栈中使用的语言、数据结构、使用方式没有强制规定，虚拟机可自有实现。
•占用的内存区大小是不固定的，可根据需要动态扩展。
程序计数器
•一块较小的内存空间，线程私有，存储当前线程执行的字节码行号指示器。
•字节码解释器通过改变这个计数器的值来选取下一条需要执行的字节码指令：分支、循环、跳转等。
•每个线程都有一个独立的程序计数器
•唯一一个在java虚拟机中不会OOM的区域



Android内存管理机制回顾-----内存模型例子




```
public class SxtStu {
    int id;
    int age;
    String sname;

    public void study(){
        System.out.println("学习");
    }

    public void kickball(){
        System.out.println("踢球");
    }

    public static void main(String[] args) {
        SxtStu s1=new SxtStu();
        System.out.println(s1.id);
        System.out.println(s1.sname);
        s1.id = 1001;
        s1.sname = "高淇";
        System.out.println(s1.id);
        System.out.println(s1.sname);
    }
}
```

第三部分：内存占用分析

2、如何看内存?

1、adb shell dumpsys meminfo com.hihonor.hiboard
2、adb shell showmap pid

```
** MEMINFO in pid 11825 [com.hihonor.hiboard] **
                   Pss  Private  Private  SwapPss      Rss     Heap     Heap     Heap
                 Total    Dirty    Clean    Dirty    Total     Size    Alloc     Free
                ------   ------   ------   ------   ------   ------   ------   ------
  Native Heap    21062    21012        0       40    23156    34528    21252     8606
  Dalvik Heap    11983     9252        4       50    31000    31618     7042    24576
Dalvik Other     29015    23608     3280        1    34212
       Stack      1636     1636        0        0     1648
     Ashmem          5        0        0        0      168
    Gfx dev        884      884        0        0      888
  Other dev         46        0       44        0      300
   .so mmap       1491      280       80        5    15196
   .jar mmap      1318        0       20        0    22756
   .apk mmap      2197        0     2144        0     2700
   .ttf mmap       140        0       52        0      360
   .dex mmap     47112    47112        0        0    47180
   .oat mmap         1        0        0        0      172
   .art mmap      1321     1232        4        1     3004
  Other mmap       146        4      128        0      716
   GL mtrack       716      716        0        0      716
     Unknown       620      584        0        2     1152
       TOTAL    119792   106320     5756       99   185324    66146    28294    33182

App Summary
                       Pss(KB)                        Rss(KB)
                       ------                         ------
            Java Heap:    10488                          34004
          Native Heap:    21012                          23156
                 Code:    49728                          90308
                Stack:     1636                           1648
             Graphics:     1600                           1604
        Private Other:    27612
               System:     7716
              Unknown:                                   34604

           TOTAL PSS:   119792            TOTAL RSS:   185324       TOTAL SWAP PSS:       99
```

- 常驻内存大小 (RSS)：应用使用的共享和非共享页面的数量
- 按比例分摊的内存大小 (PSS)：应用使用的非共享页面的数量加上共享页面的均匀分摊数量（例如，如果三个进程共享 3MB，则每个进程的 PSS 为 1MB）
- 独占内存大小 (USS)：应用使用的非共享页面数量（不包括共享页面）

内存指标：PSS

背景知识

```
** MEMINFO in pid 12162 [com.hihonor.hiboard] **
                   Pss  Private  Private  SwapPss      Rss     Heap     Heap     Heap
                 Total    Dirty    Clean    Dirty    Total     Size    Alloc     Free
                ------   ------   ------   ------   ------   ------   ------   ------
  Native Heap   207018   206964        0      886   208988   286840   140374   139509
  Dalvik Heap    25392    20932     1704       30    42912    65523    16381    49142
Dalvik Other     25456    13512     5988       53    37676
       Stack      5364     5364        0       72     5372
      Ashmem       548      520        0        0      908
    Gfx dev       9180     9180        0        0     9184
  Other dev         49        0       40        0      492
   .so mmap       5395     1044      176       11    37304
   .jar mmap      3376        0      224        0    34116
   .apk mmap     10955        0     2768        0    24724
   .ttf mmap      1179        0      308        0     3284
   .dex mmap     30020     1128    20632        0    42020
   .oat mmap       128        0        0        0     4192
   .art mmap       776      668        4        1     2996
  Other mmap       497        4      288        0     1608
   GL mtrack      1216     1216        0        0     1216
     Unknown      2918     2904        0        7     3340
       TOTAL    330527   263436    32132     1060   460332   352363   156755   188651

App Summary
                       Pss(KB)                        Rss(KB)
                       ------                         ------
           Java Heap:    21604                          45908
         Native Heap:   206964                         208988
                Code:    26324                         152600
               Stack:     5364                           5372
            Graphics:    10396                          10400
       Private Other:    24916
              System:    34959
             Unknown:                                   37064

         TOTAL PSS:   330527            TOTAL RSS:   460332       TOTAL SWAP PSS:     1060
```

Java Heap:
Dalvik Heap private dirty+ .art mmap private (clean+ dirty) = 20932 + 4 + 668 = **21604**

Native Heap:
Native private dirty = 206964

Code:
.so mmap private (clean + dirty) + .jar mmap private (clean + dirty) + .apk mmap private (clean + dirty) + .ttf mmap private (clean+ dirty) + .dex mmap private (clean + dirty) + .oat mmap private (clean + dirty) = (1044 + 176) + (0+ 224) + (0 + 2768) + (0 + 308) + (1128 + 20632) + (0 + 0) = **26324**

Stack:
Stack private dirty = 5346 （单个线程最多64KB x100=6M?）

Graphics:
GL mtrack private (clean + dirty) + EGL **mtrack** private(clean + dirty) = (1216 + 0) + (9180 + 0) = 10396

Private other:
TOTAL private (clean + dirty) - Java Heap - Native Heap- Code- Stack - Graphic = (263436 + 32132) - 21604 - 206964 - 26324 - 5346 - 10396 = 24916

System :
TOTAL - TOTAL private (clean + dirty) = 330527 - (263436 + 32132) = 34959

内存脚本测试的时候不使用包名，带包名会触发GC

App summary分类中内存space类型PSS计算方法

frameworks/base/core/java/android/os/Debug.java
frameworks/base/core/jni/android_os_Debug.cpp

这时候，我们就要参考源码，探求数据来源集合的真像。在Debug.java里，我们看到：

```
@UnsupportedAppUsage
public int getSummaryJavaHeap() {
    return dalvikPrivateDirty + getOtherPrivate(OTHER_ART);
}

public int getSummaryNativeHeap() {
    return nativePrivateDirty;
}
@UnsupportedAppUsage
public int getSummaryCode() {
    return getOtherPrivate(OTHER_SO)
      + getOtherPrivate(OTHER_JAR)
      + getOtherPrivate(OTHER_APK)
      + getOtherPrivate(OTHER_TTF)
      + getOtherPrivate(OTHER_DEX)
        + getOtherPrivate(OTHER_OAT)
        + getOtherPrivate(OTHER_DALVIK_OTHER_ZYGOTE_CODE_CACHE)
        + getOtherPrivate(OTHER_DALVIK_OTHER_APP_CODE_CACHE);
}
@UnsupportedAppUsage
public int getSummaryStack() {
    return getOtherPrivateDirty(OTHER_STACK);
}
@UnsupportedAppUsage
public int getSummarySystem() {
    return getTotalPss()
      - getTotalPrivateClean()
      - getTotalPrivateDirty();
}
```

于是，根据源码绘制右图表。根据右边，我们能知道Java Heap数据到底是怎么来的。
（读者需具备脏页回写、匿名页、swap、malloc/mmap等相关基础概念才能更好理解）

但现在有个难题，就是例如Dalvik Heap是哪些文件？Native Heap是哪些文件？
.Heap又是哪些文件？
这个时候我们就需要一个“内存类型对应内存对象”的表格，来知道具体的匹配规则。

dumpsys meminfo -a –local packageName各Summary分类内存数据来源图

App Summary分类PSS计算方法				
summary分类	space类型			
Pss(KB)	Private Dirty		Private Clean	
Java Heap	Dalvik Heap	.Heap		
		.LOS		
		.Zygote		
		.NonMoving		
	.art mmap		.art mmap	
Native Heap:	Native Heap			
Code:	.so mmap		.so mmap	
	.jar mmap		.jar mmap	
	.apk mmap		.apk mmap	
	.ttf mmap		.ttf mmap	
	.dex mmap		.dex mmap	
	.oat mmap		.oat mmap	
	.ZygoteJIT		.ZygoteJIT	
	.AppJIT		.AppJIT	
Stack:	Stack			
Graphics:	Gfx dev		Gfx dev	
	EGL mtrack		EGL mtrack	
	GL mtrack		GL mtrack	
Private Other:	Private Dirty + Private Clean -Java Heap - Native Heap - code - stack - Graphics			
System:	Pss Total - Private Dirty - Private Clean			
Unknown:	/			

结合第10页能更好理解此图

frameworks/base/core/java/android/os/Debug.java

frameworks/base/core/jni/android_os_Debug.cpp

meninfo对应showmap原理表

内存space类型	showmap指令里的space（匹配规则）
Native Heap	base::StartsWith(name, "[heap]") base::StartsWith(name, "[anon:libc_malloc]") base::StartsWith(name, "[anon:scudo:") base::StartsWith(name, "[anon:GWP-ASan")
Dalvik Heap	base::StartsWith(name, "[anon:dalvik-alloc space") base::StartsWith(name, "[anon:dalvik-main space") base::StartsWith(name, "[anon:dalvik-large object space") base::StartsWith(name, "[anon:dalvik-free list large object space") base::StartsWith(name, "[anon:dalvik-non moving space") base::StartsWith(name, "[anon:dalvik-zygote space")
Dalvik Other	base::StartsWith(name, "/dev/ashmem/jit-zygote-cache") base::StartsWith(name, "/memfd:jit-cache") base::StartsWith(name, "/memfd:jit-zygote-cache") base::StartsWith(name, "[anon:dalvik-")列表中剔除“Dalvik Heap”后剩余的部分
Stack	base::StartsWith(name, "[stack") base::StartsWith(name, "[anon:stack_and_tls:")
Ashmem	base::StartsWith(name, "/dev/ashmem")
Gfx dev	base::StartsWith(name, "/dev/kgsl-3d0")
Other dev	base::StartsWith(name, "/dev/")列表剔除下面后剩余的部分 剔除base::StartsWith(name, "/dev/kgsl-3d0") 剔除base::StartsWith(name, "/dev/ashmem/CursorWindow") 剔除base::StartsWith(name, "/dev/ashmem/jit-zygote-cache") 剔除base::StartsWith(name, "/dev/ashmem")
.so mmap	base::EndsWith(name, ".so") vma.start == prev_end && prev_heap == HEAP_SO // bss section of a shared library
.jar mmap	base::EndsWith(name, ".jar")
.apk mmap	base::EndsWith(name, ".apk")
.ttf mmap	base::EndsWith(name, ".ttf")
.dex mmap	base::EndsWith(name, ".odex") namesz > 4 && strstr(name.c_str(), ".dex") base::EndsWith(name, ".vdex")
.oat mmap	base::EndsWith(name, ".oat")
.art mmap	base::EndsWith(name, ".art") base::EndsWith(name, ".art]")
Other mmap	showmap列表，剔除上面所有后剩余的部分。
EGL mtrack	gralloc分配的显存，/proc/pid/smaps里不体现，主要通过libmemtrack来获取。 窗口系统、SurfaceView/TextureView和其他的由gralloc分配的GraphicBuffer总和
GL mtrack	驱动上报的GL内存使用情况。/proc/pid/smaps里不体现，主要通过libmemtrack来获取。主要是GL texture大小，GL command buffer，固定的全局驱动程序RAM开销等的总和
Unknown	gpu使用内存剩余部分。/proc/pid/smaps里不体现，主要通过libmemtrack来获取。
TOTAL	

Dalvik Details		
.Heap	base::StartsWith(name, "[anon:dalvik-alloc space") base::StartsWith(name, "[anon:dalvik-main space")	等于Dalvik Heap
.LOS	base::StartsWith(name, "[anon:dalvik-large object space") base::StartsWith(name, "[anon:dalvik-free list large object space")	
.Zygote	base::StartsWith(name, "[anon:dalvik-zygote space")	
.NonMoving	base::StartsWith(name, "[anon:dalvik-non moving space")	
.LinearAlloc	base::StartsWith(name, "[anon:dalvik-LinearAlloc")	等于Dalvik Other（其中.zygoteJIt、.AppJIT也属于code部分。换句话说Code包含了Dalvik other的一部分）
.GC	base::StartsWith(name, "[anon:dalvik-")列表中剔除其他类型后剩下部分	
.ZygoteJIT	base::StartsWith(name, "/dev/ashmem/jit-zygote-cache") base::StartsWith(name, "/memfd:jit-zygote-cache")	
.AppJIT	base::StartsWith(name, "/memfd:jit-cache") base::StartsWith(name, "[anon:dalvik-data-code-cache") base::StartsWith(name, "[anon:dalvik-jit-code-cache")	
.IndirectRef	base::StartsWith(name, "[anon:dalvik-indirect ref")	
.Boot vdex	base::EndsWith(name, ".vdex") && strstr(name.c_str(), "@boot") base::EndsWith(name, ".vdex") && strstr(name.c_str(), "/boot") base::EndsWith(name, ".vdex") && strstr(name.c_str(), "/apex")	
.App dex	base::EndsWith(name, ".odex") namesz > 4 && strstr(name.c_str(), ".dex"	
.App vdex	base::EndsWith(name, ".vdex")列表去除".Boot vdex"后剩余的部分	
.Boot art	base::EndsWith(name, ".art") && strstr(name.c_str(), "@boot") base::EndsWith(name, ".art") && strstr(name.c_str(), "/boot") base::EndsWith(name, ".art") && strstr(name.c_str(), "/apex")	

一定要区分三个概念！
Summary分类、space类型、space

debug包前台内存分布-模块明细

frameworks/base/core/java/android/os/Debug.java

```
@UnsupportedAppUsage
public int getSummaryJavaHeap() {
    return dalvikPrivateDirty + getOtherPrivate(OTHER_ART);
}


@UnsupportedAppUsage
public int getSummaryNativeHeap() {
    return nativePrivateDirty;
}

@UnsupportedAppUsage
public int getSummaryCode() {
    return getOtherPrivate(OTHER_SO)
      + getOtherPrivate(OTHER_JAR)
      + getOtherPrivate(OTHER_APK)
      + getOtherPrivate(OTHER_TTF)
      + getOtherPrivate(OTHER_DEX)
        + getOtherPrivate(OTHER_OAT)
        + getOtherPrivate(OTHER_DALVIK_OTHER_ZYGOTE_CODE_CACHE)
        + getOtherPrivate(OTHER_DALVIK_OTHER_APP_CODE_CACHE);
}


@UnsupportedAppUsage
public int getSummaryStack() {
    return getOtherPrivateDirty(OTHER_STACK);
}


@UnsupportedAppUsage
public int getSummarySystem() {
    return getTotalPss()
      - getTotalPrivateClean()
      - getTotalPrivateDirty();
}
```

1	HNPGT:/ # dumpsys meminfo -a --local com.hihonor.id											
2	Applications M	mory Usa	ge (in K	ilobytes)	:							
3	Uptime: 5452	Realtim	e: 17042	802								
4												
5	** MEMINFO i	id 23322	[com.hi	honor.id]	**							
6		**Pss**	Pss	Shared	**Private**	Shared	**Private**	SwapPss	Rss	Heap	Heap	Heap
7		**Total**	Clean	Dirty	**Dirty**	Clean	**Clean**	Dirty	Total	Size	Alloc	Free
8		------	------	------	------	------	------	------	------	------	------	------
9	Native Heap	28011	0	68	27960	1320	0	18	29348	0	0	0
10	Dalvik Heap	8832	0	592	8760	1116	4	81	10472	0	0	0
11	Dalvik Other	6635	0	3884	4628	1340	0	0	9852			
12	Stack	3220	0	0	3220	12	0	0	3232			
13	Ashmem	14	0	460	0	0	0	0	460			
14	Gfx dev	7576	0	0	6592	4	984	0	7580			
15	Other dev	149	0	484	0	0	132	0	616			
16	.so mmap	4268	460	44	668	29376	460	0	30548			
17	.jar mmap	6145	712	0	0	33532	712	0	34244			
18	.apk mmap	2057	664	0	0	8072	664	0	8736			
19	.ttf mmap	320	96	0	0	844	96	0	940			
20	.dex mmap	41294	0	0	41260	636	0	0	41896			
21	.oat mmap	62	16	12	0	660	16	0	688			
22	.art mmap	7860	64	1764	7084	8936	64	37	17848			
23	Other mmap	95	0	0	24	740	24	0	788			
24	EGL mtrack	60176	0	0	60176	0	0	0	60176			
25	GL mtrack	4596	0	0	4596	0	0	0	4596			
26	Unknown	881	0	104	860	508	4	1	1476			
27	TOTAL	182328	2012	7412	165828	87096	3160	137	263496	0		
28												
29	Dalvik Details											
30	.Heap	6600	0	0	6600	0	0	0	6600			
31	.LOS	525	0	0	520	36	0	19	556			
32	.Zygote	1159	0	592	1092	1080	4	62	2768			
33	.NonMoving	548	0	0	548	0	0	0	548			
34	.LinearAlloc	4646	0	528	4304	1264	0	0	6096			
35	.GC	249	0	0	248	76	0	0	324			
36	.ZygoteJIT	0	0	28	0	0	0	0	28			
37	.AppJIT	1676	0	3328	12	0	0	0	3340			
38	.IndirectRef	64	0	0	64	0	0	0	64			
39	.Boot vdex	33	0	0	0	596	0	0	596			
40	.App dex	41260	0	0	41260	28	0	0	41288			
41	.App vdex	1	0	0	0	12	0	0	12			
42	.Boot art	7860	64	1764	7084	8936	64	37	17848			
43												
44	App Summary											
45		Pss(KB)		Rss(KB)								
46		------		------								
47	Java Heap:	15908		28320								
48	Native Heap:	27960		29348								
49	Code:	43888		120420								
50	Stack:	3220		3232								
51	Graphics:	72348		72352								
52	Private Other:	5664										
53	System:	13340										
54	Unknown:			9824								
55												
56	TOTAL PSS:	182328	TOTAL RSS	263496	TOTAL SW	137						

类型	匹配字符串
Ashmem	/dev/ashmem
Gfx dev	/dev/kgsl-3d0
Other dev	/dev/*
.so mmap	*.so 例 /system/lib64/libstdc++.so
.jar mmap	*.jar 例 /system/framework/framework.jar
.apk mmap	*.apk 例 /system/framework/framework-res.apk
.ttf mmap	*.ttf 例 /system/fonts/Roboto-BlackItalic.ttf
.dex mmap	*.vdex 例 /system/framework/boot.vdex
.oat mmap	*.oat 例 /system/framework/arm64/boot-ext.oat
.art mmap	*.art 例 /system/framework/arm64/boot.art
Other mmap	不属于上面的类型

手动验证						
Java Heap:	15908	8760	.Heap	.LOS	.Zygote	.NonMoving
Native Heap:	27960					
Code:	43888					
Stack:	3220					
Graphics:	72348					
Private Other:	可以认为是私有内存那两列无颜色的部分					
System:	13340 (A-B-C)					
Unknown:						

注意：用excel打开txt即可快速排版

应用内存分析脚本化工具

smaps_parser.py

方法一：
python smaps_parser.py -p 21936 -o out.txt

方法二：
1. 获取对应进程的 smaps 文件```adb pull /proc/pid_of_app/smaps . `
2. 执行解析脚本```python /smap/smaps_parser.py -f <path_of_smaps>```

```
00008000-00028000 r--s 00000000 00:10 26665
Size:                128 kB
KernelPageSize:        4 kB
MMUPageSize:           4 kB
Rss:                   0 kB
Pss:                   0 kB
Shared_Clean:          0 kB
Shared_Dirty:          0 kB
Private_Clean:         0 kB
Private_Dirty:         0 kB
Referenced:            0 kB
Anonymous:             0 kB
LazyFree:              0 kB
AnonHugePages:         0 kB
ShmemPmdMapped:        0 kB
Shared_Hugetlb:        0 kB
Private_Hugetlb:       0 kB
Swap:                  0 kB
SwapPss:               0 kB
Locked:                0 kB
VmFlags: rd mr me ms
00028000-00029000 ---p 00000000 00:00 0
Size:                  4 kB
KernelPageSize:        4 kB
MMUPageSize:           4 kB
Rss:                   0 kB
Pss:                   0 kB
Shared_Clean:          0 kB
Shared_Dirty:          0 kB
Private_Clean:         0 kB
Private_Dirty:         0 kB
Referenced:            0 kB
Anonymous:             0 kB
LazyFree:              0 kB
AnonHugePages:         0 kB
ShmemPmdMapped:        0 kB
Shared_Hugetlb:        0 kB
Private_Hugetlb:       0 kB
Swap:                  0 kB
SwapPss:               0 kB
Locked:                0 kB
VmFlags: mr mw me nr
```

```
** MEMINFO in pid 4305 [com.zhihu.android] **
                   Pss  Private  Private     Swap     Heap     Heap     Heap
                 Total    Dirty    Clean    Dirty     Size    Alloc     Free
                ------   ------   ------   ------   ------   ------   ------
  Native Heap    71776    71700        0        0    96768    90945     5822
  Dalvik Heap    32538    32472        0        0    38843    19422    19421
 Dalvik Other     2825     2820        0        0
       Stack       36       36        0        0
       Ashmem      90        4        0        0
       Gfx dev  30048    30048        0        0
     Other dev    170        0      168        0
      .so mmap  15902      980     8876        0
      .jar mmap  2130        0      980        0
      .apk mmap 15711      132     6604        0
      .ttf mmap  2962        0      416        0
      .dex mmap 30974       56    19344        0
      .oat mmap  1051        0        4        0
      .art mmap 12847    12544        0        0
     Other mmap  1620      108      604        0
     EGL mtrack 30828    30828        0        0
      GL mtrack 16928    16928        0        0
        Unknown  5640     5636        0        0
          TOTAL 274076   204292    36996        0   135611   110367    25243
```

```
Unknown : 1.609 M
    pss: 1.609 M
    swapPss: 0.000 M
        : 716 kB
        [anon:.bss] : 465 kB
        [anon:linker_alloc] : 280 kB
        [anon:bionic_alloc_small_objects] : 112 kB
        [anon:System property context nodes] : 12 kB
        [anon:arc4random data] : 8 kB
        [anon:cfi shadow] : 8 kB
        [anon:atexit handlers] : 4 kB
        [anon:bionic_alloc_lob] : 4 kB
Dalvik : 7.269 M
    pss: 7.269 M
    swapPss: 0.000 M
        [anon:dalvik-main space (region space)] : 6180 kB
        [anon:dalvik-zygote space] : 496 kB
        [anon:dalvik-non moving space] : 440 kB
        [anon:dalvik-free list large object space] : 153 kB
Native : 32.578 M
    pss: 32.578 M
    swapPss: 0.000 M
        [anon:libc_malloc] : 32578 kB
Dalvik Other : 1.969 M
    pss: 1.969 M
    swapPss: 0.000 M
        [anon:dalvik-LinearAlloc] : 1453 kB
        [anon:dalvik-indirect ref table] : 136 kB
        [anon:dalvik-large marked objects] : 128 kB
        [anon:dalvik-region space live bitmap] : 92 kB
        [anon:dalvik-card table] : 84 kB
        [anon:dalvik-Jit thread pool worker thread 0] : 24 kB
        [anon:dalvik-large object free list space allocation info map] : 16 kB
        [anon:dalvik-allocspace non moving space live-bitmap 1] : 8 kB
        [anon:dalvik-live stack] : 4 kB
        [anon:dalvik-mod union bitmap] : 4 kB
        [anon:dalvik-Runtime worker thread 0] : 4 kB
        [anon:dalvik-zygote-data-code-cache] : 4 kB
        [anon:dalvik-Runtime worker thread 3] : 4 kB
        [anon:dalvik-Runtime worker thread 2] : 4 kB
        [anon:dalvik-Runtime worker thread 1] : 4 kB
Stack : 0.048 M
    pss: 0.048 M
    swapPss: 0.000 M
        [stack] : 48 kB
Cursor : 0.000 M
    pss: 0.000 M
    swapPss: 0.000 M
Ashmem : 0.064 M
    pss: 0.064 M
    swapPss: 0.000 M
        /dev/ashmem/MemoryHeapBase (deleted) : 60 kB
        /dev/ashmem/RemoteDataSource (deleted) : 2 kB
        /dev/ashmem/GFXStats-12747 (deleted) : 2 kB
Gfx dev : 3.616 M
    pss: 3.616 M
    swapPss: 0.000 M
        /dev/kgsl-3d0 : 3616 kB
Other dev : 0.168 M
    pss: 0.168 M
    swapPss: 0.000 M
        /dev/binder : 160 kB
        /dev/hwbinder : 8 kB
```

2、如何看内存？（event logs）

```
07-10 05:08:36.944  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,338936832,226566144,93021184,371167232,0,3,18]
07-10 06:08:39.695  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,345115648,233213952,92550144,377716736,0,3,25]
07-10 07:08:42.569  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,350891008,238882816,92660736,383381504,2,3,19]
07-10 08:08:43.018  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,356528128,243150848,94085120,387555328,0,3,23]
07-10 09:08:43.113  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,298388480,179765248,99550208,323301376,0,3,21]
07-10 10:08:43.269  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,302954496,181706752,102583296,324182016,2,3,21]
07-10 11:08:43.383  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,314923008,186605568,109778944,329023488,0,3,19]
07-10 12:08:43.534  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,321823744,187977728,115231744,330620928,0,3,23]
07-10 13:08:43.692  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,352488448,195100672,138179584,336982016,2,3,24]
07-10 14:08:43.766  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,358299648,195166208,143868928,336916480,0,3,19]
07-10 14:55:16.106  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,330835968,212443136,97209344,359677952,1,3,24]
07-10 15:55:22.737  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,292997120,173805568,97819648,320565248,0,3,24]
07-10 16:08:44.047  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,294519808,175296512,97815552,322080768,2,3,23]
07-10 17:08:44.138  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,305504256,185126912,99143680,332021760,0,3,17]
07-10 18:08:44.280  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,309027840,184291328,104863744,327696384,0,3,19]
07-10 19:08:44.410  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,320617472,188379136,113029120,330657792,2,3,19]
07-10 20:08:44.534  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,326697984,194154496,113171456,336183296,0,3,19]
07-10 21:08:44.697  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,330041344,186281984,125053952,326279168,0,3,23]
07-10 22:08:44.828  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,336671744,192270336,126192640,331141120,2,3,24]
07-10 23:08:44.935  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,286224384,142061568,125959168,280936448,0,3,22]
07-11 00:08:45.066  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,292622336,148303872,126090240,287191040,0,3,22]
07-11 01:08:45.185  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,298168320,152276992,127758336,290701312,2,3,22]
07-11 02:08:45.295  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,303842304,157769728,127979520,296398848,0,3,21]
07-11 03:08:45.406  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,309489664,162680832,128724992,301211648,0,3,22]
07-11 04:08:45.523  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,315240448,167858176,129271808,306388992,2,3,18]
07-11 05:08:45.646  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,320703488,173113344,129640448,311226368,0,3,23]
07-11 06:08:45.763  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,326428672,178548736,129910784,316850176,0,3,23]
07-11 07:08:45.897  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,332383232,184532992,129869824,322842624,2,3,24]
07-11 08:08:46.018  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,337946624,189681664,130347008,327925760,0,3,24]
07-11 09:08:46.116  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,288368640,138670080,131962880,276955136,0,3,17]
07-11 10:08:46.363  2440  2831 I am_pss  : [31934,10031,com.hihonor.hiboard,279811072,145600512,111417344,284737536,2,3,18]
```

格式：（Name|type|unit）

- Name：表示这个字段的意义
- Type：表示这个字段的数据格式，取值为：
  1: int
  2: long
  3: string
  4: list
  5: float
- unit: 取值为
  1: Number of objects
  2: Number of bytes
  3: Number of milliseconds
  4: Number of allocations
  5: Id
  6: Percent
  s: Number of seconds (monotonic time)

am_pss (Pid|1|5),(UID|1|5),(Process Name|3),(Pss|2|2),(Uss|2|2),(SwapPss|2|2),(Rss|2|2),(StatType|1|5),(ProcState|1|5),(TimeToCollect|2|2)

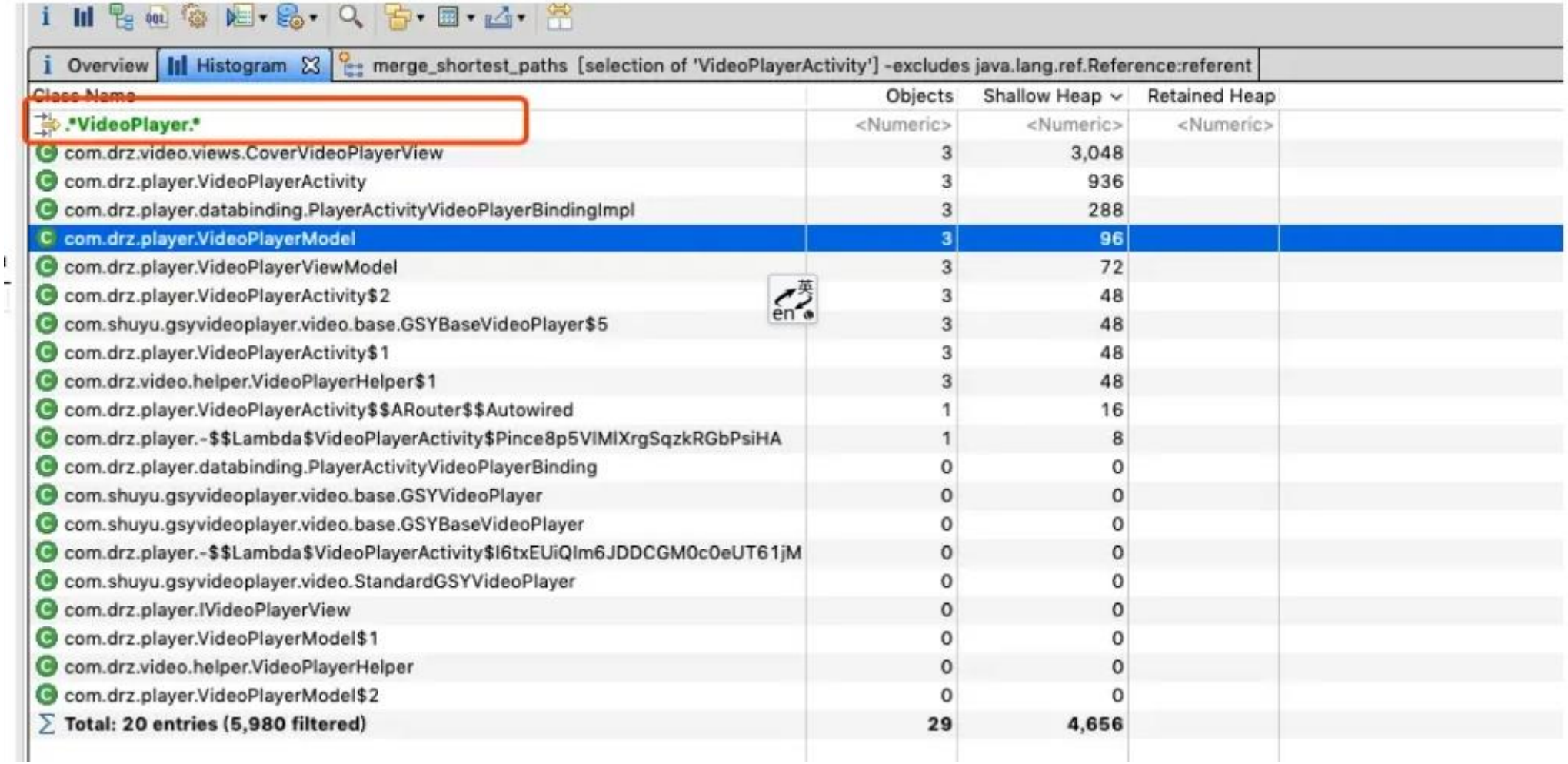
services/core/java/com/android/server/am/EventLogTags.logtags - platform/frameworks/base - Git at Google (googlesource.com)

2、如何看内存？MAT的使用

adb shell am dumpheap %PKG_NAME%
/data/local/tmp/%PKG_NAME%.hprof

hprof-conv.exe

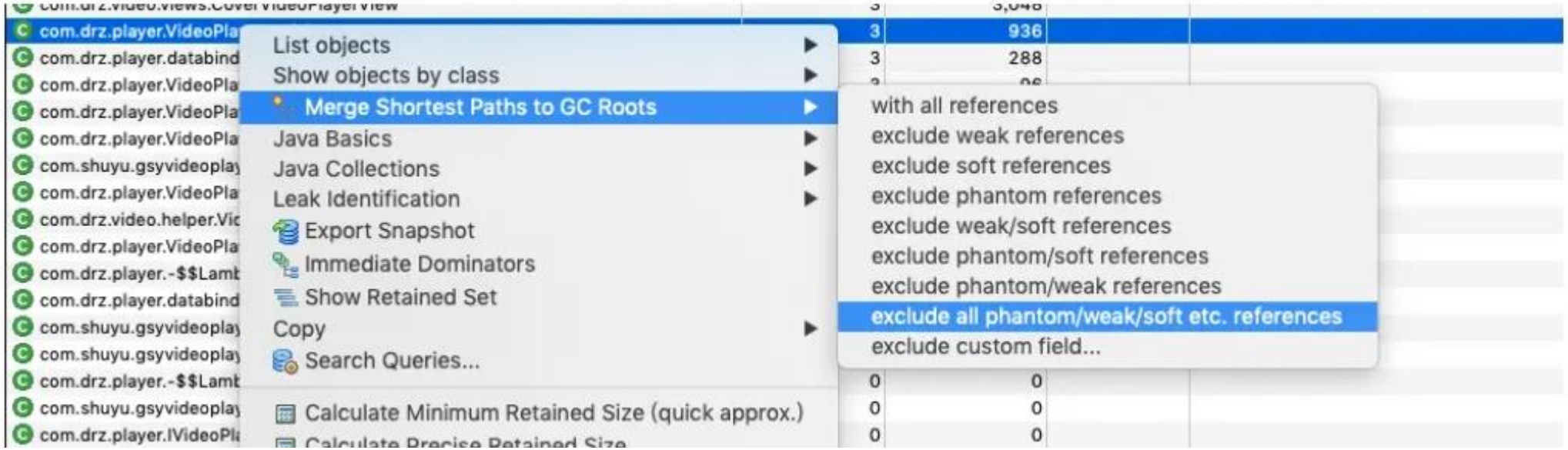
profile 和 MAT工具



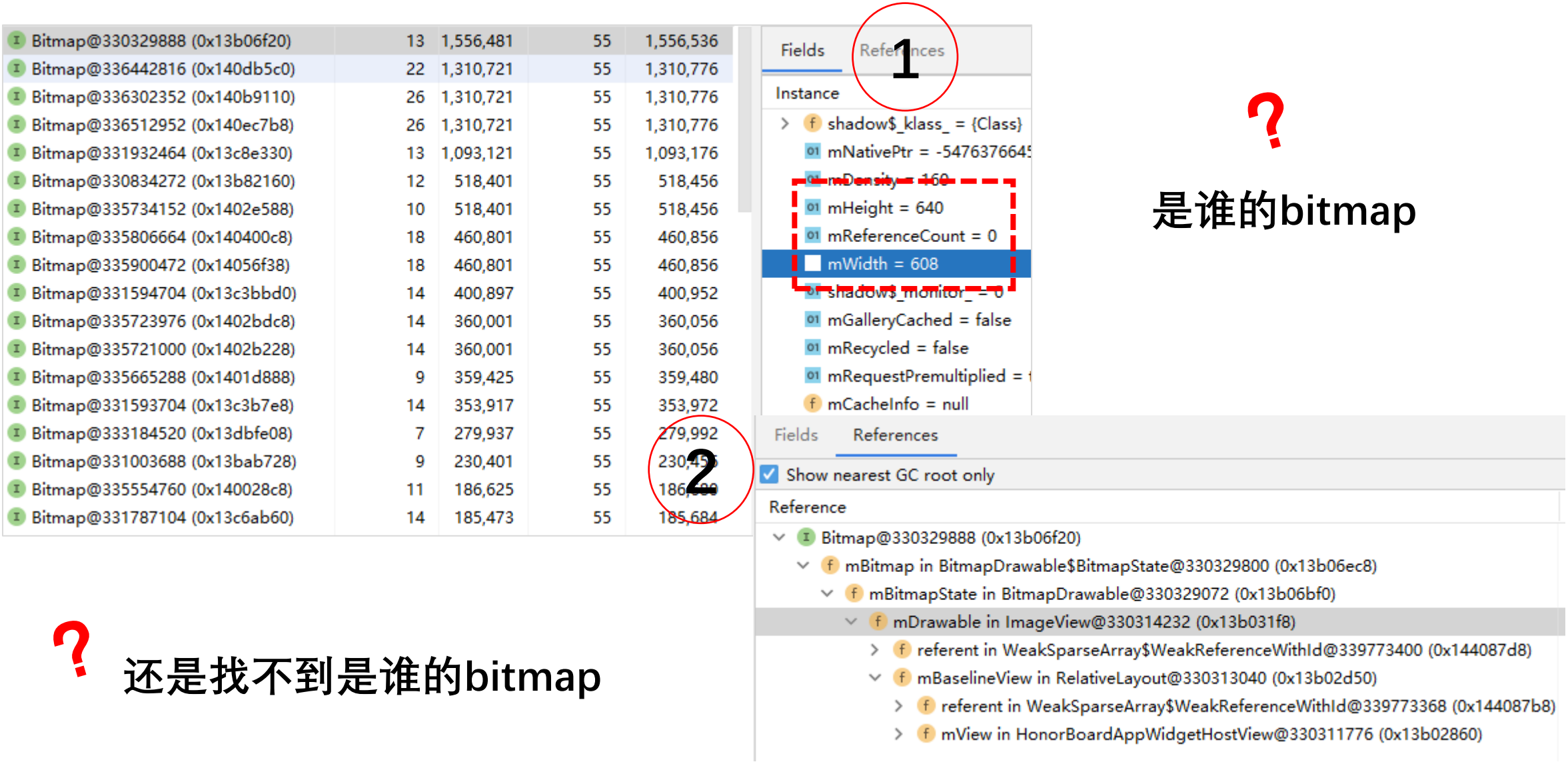
Overview | Histogram | merge_shortest_paths [selection of 'VideoPlayerActivity'] -excludes java.lang.ref.Reference:referent

Class Name	Objects	Shallow Heap	Retained Heap
.*VideoPlayer.*	<Numeric>	<Numeric>	<Numeric>
com.drz.video.views.CoverVideoPlayerView	3	3,048	
com.drz.player.VideoPlayerActivity	3	936	
com.drz.player.databinding.PlayerActivityVideoPlayerBindingImpl	3	288	
com.drz.player.VideoPlayerModel	3	96	
com.drz.player.VideoPlayerViewModel	3	72	
com.drz.player.VideoPlayerActivity$2	3	48	
com.shuyu.gsyvideoplayer.video.base.GSYBaseVideoPlayer$5	3	48	
com.drz.player.VideoPlayerActivity$1	3	48	
com.drz.video.helper.VideoPlayerHelper$1	3	48	
com.drz.player.VideoPlayerActivity$$ARouter$$Autowired	1	16	
com.drz.player.-$$Lambda$VideoPlayerActivity$Pince8p5VlMlXrgSqzkRGbPsiHA	1	8	
com.drz.player.databinding.PlayerActivityVideoPlayerBinding	0	0	
com.shuyu.gsyvideoplayer.video.base.GSYVideoPlayer	0	0	
com.shuyu.gsyvideoplayer.video.base.GSYBaseVideoPlayer	0	0	
com.drz.player.-$$Lambda$VideoPlayerActivity$I6txEUiQIm6JDDCGM0c0eUT61jM	0	0	
com.shuyu.gsyvideoplayer.video.StandardGSYVideoPlayer	0	0	
com.drz.player.IVideoPlayerView	0	0	
com.drz.player.VideoPlayerModel$1	0	0	
com.drz.video.helper.VideoPlayerHelper	0	0	
com.drz.player.VideoPlayerModel$2	0	0	
Total: 20 entries (5,980 filtered)	**29**	**4,656**	

profile 和 MAT工具



Class Name	Ref. Objects	Shallow Heap	Ref. Shallow Heap	Retained Heap
<Regex>	<Numeric>	<Numeric>	<Numeric>	<Numeric>
android.app.ActivityThread @ 0x12c04240 Unknown	3	192	936	11,224
mBoundApplication android.app.ActivityThread$AppBindData @ 0x12cb6560	3	64	936	240
info android.app.LoadedApk @ 0x12ca8b30	3	104	936	464
mReceivers android.util.ArrayMap @ 0x12cbaa80	3	24	936	256
mArray java.lang.Object[8] @ 0x131a14f0	3	48	936	200
[1] android.util.ArrayMap @ 0x131a0d40	3	24	936	152
mArray java.lang.Object[16] @ 0x13a93060	3	80	936	80
[2] com.shuyu.gsyvideoplayer.utils.NetInfoModule$ConnectivityBroadcastReceiver @ 0x13900b20	1	24	312	80
this$0 com.shuyu.gsyvideoplayer.utils.NetInfoModule @ 0x13900a80	1	32	312	48
[4] com.shuyu.gsyvideoplayer.utils.NetInfoModule$ConnectivityBroadcastReceiver @ 0x12ff4ce0	1	24	312	80
this$0 com.shuyu.gsyvideoplayer.utils.NetInfoModule @ 0x12ff4b20	1	32	312	48
[0] com.shuyu.gsyvideoplayer.utils.NetInfoModule$ConnectivityBroadcastReceiver @ 0x13780be0	1	24	312	80
this$0 com.shuyu.gsyvideoplayer.utils.NetInfoModule @ 0x13780bc0	1	32	312	48
mNetChangeListener com.shuyu.gsyvideoplayer.video.base.GSYVideoView$4 @ 0x139c1480	1	16	312	16
this$0 com.drz.video.views.CoverVideoPlayerView @ 0x13129800	1	1,016	312	1,960
mContext com.drz.player.VideoPlayerActivity @ 0x12d17580	1	312	312	9,368

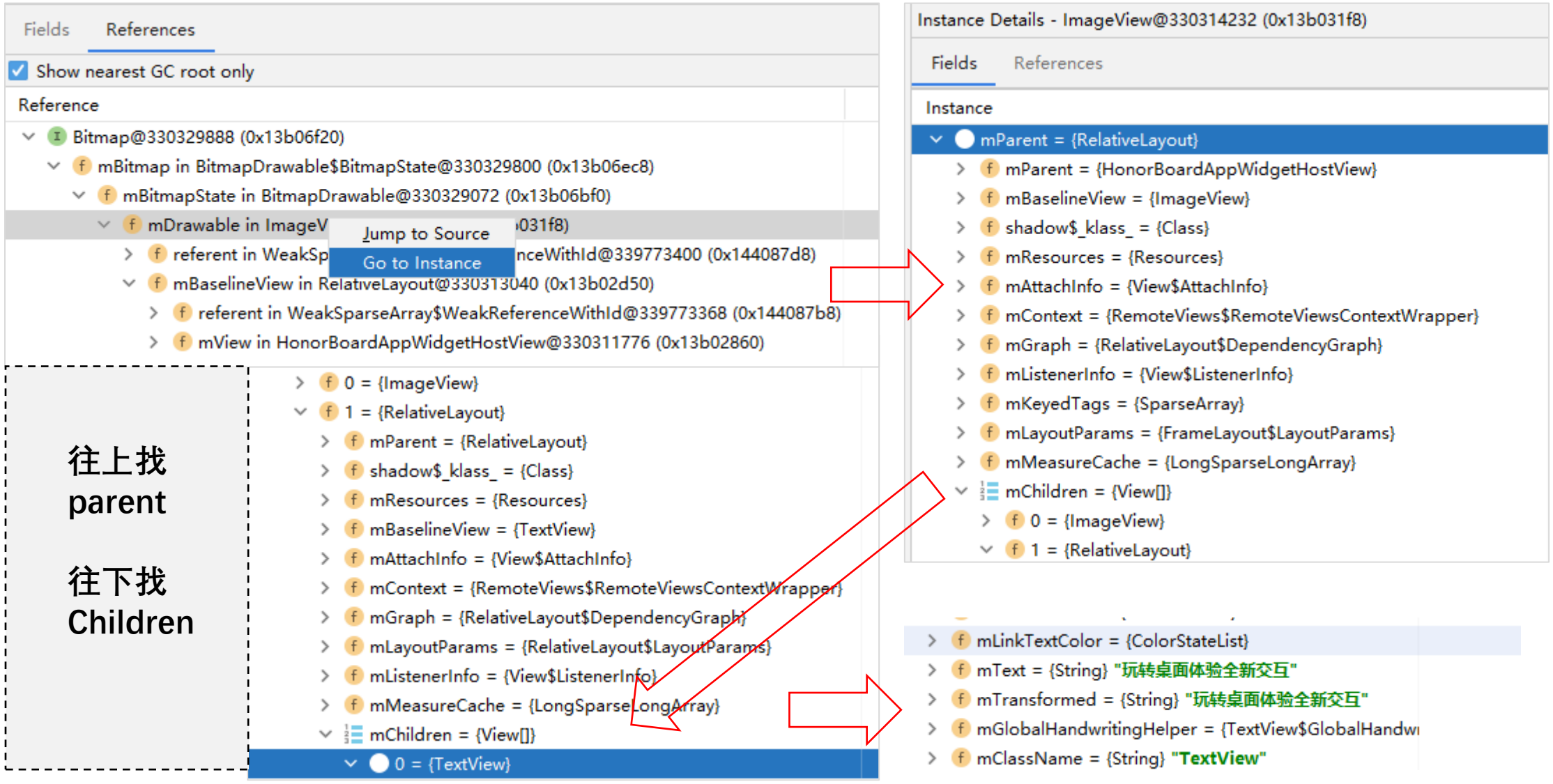


?

是谁的bitmap

?

还是找不到是谁的bitmap


Fields
References
Show nearest GC root only
Reference
Bitmap@330329888 (0x13b06f20)
mBitmap in BitmapDrawable$BitmapState@330329800 (0x13b06ec8)
mBitmapState in BitmapDrawable@330329072 (0x13b06bf0)
mDrawable in ImageV
031f8)
Jump to Source
Go to Instance
referent in WeakSp
nceWithId@339773400 (0x144087d8)
mBaselineView in RelativeLayout@330313040 (0x13b02d50)
referent in WeakSparseArray$WeakReferenceWithId@339773368 (0x144087b8)
mView in HonorBoardAppWidgetHostView@330311776 (0x13b02860)
往上找
parent
往下找
Children
0 = {ImageView}
1 = {RelativeLayout}
mParent = {RelativeLayout}
shadow$_klass_ = {Class}
mResources = {Resources}
mBaselineView = {TextView}
mAttachInfo = {View$AttachInfo}
mContext = {RemoteViews$RemoteViewsContextWrapper}
mGraph = {RelativeLayout$DependencyGraph}
mLayoutParams = {RelativeLayout$LayoutParams}
mListenerInfo = {View$ListenerInfo}
mMeasureCache = {LongSparseLongArray}
mChildren = {View[]}
0 = {TextView}
Instance Details - ImageView@330314232 (0x13b031f8)
Fields
References
Instance
mParent = {RelativeLayout}
mParent = {HonorBoardAppWidgetHostView}
mBaselineView = {ImageView}
shadow$_klass_ = {Class}
mResources = {Resources}
mAttachInfo = {View$AttachInfo}
mContext = {RemoteViews$RemoteViewsContextWrapper}
mGraph = {RelativeLayout$DependencyGraph}
mListenerInfo = {View$ListenerInfo}
mKeyedTags = {SparseArray}
mLayoutParams = {FrameLayout$LayoutParams}
mMeasureCache = {LongSparseLongArray}
mChildren = {View[]}
0 = {ImageView}
1 = {RelativeLayout}
mLinkTextColor = {ColorStateList}
mText = {String} "玩转桌面体验全新交互"
mTransformed = {String} "玩转桌面体验全新交互"
mGlobalHandwritingHelper = {TextView$GlobalHandwr
mClassName = {String} "TextView"